Lab Report 3: TIM Driver — LED Blink & PWM

Authors:
J.M. Páez Sandoval, 37945
L.A. Lugo Muñiz, 37915
D. Lopez Moreno, 37984
C.A. Martínez Macías, 33369
Teacher : Carlos A. Villareal
CETYS Universidad
Facultad de Ingeniería
Curso: Microcontroladores y Sistemas Embebidos
Fecha: May 29, 2026

Abstract*—This report documents the bare-metal implementation of an ADC-based sensor-controlled PWM system on the NUCLEO-F411RE development board (STM32F411RE, ARM Cortex-M4). The project configures ADC1 in single-conversion mode to sample a potentiometer voltage on PA1 (ADC1_IN1) and maps the 12-bit ADC result (0–4095) to a PWM duty cycle (0–100%) driving TIM1 Channel 1 on PA8 at 1 kHz. A modular driver architecture separates low-level register access (adc_driver, gpio_driver, tim_driver) from application-level logic (sensor, pwm). Hardware verification confirmed linear duty-cycle scaling across the full potentiometer range.*

Keywords*—ADC,STM32F411RE, bare-metal, PWM, potentiometer, sensor, duty cycle, TIM1, NUCLEO-F411RE, embedded systems.*

I. INTRODUCTION

Analog-to-Digital Conversion (ADC) is a critical capability in embedded systems, enabling microcontrollers to measure real-world physical quantities — voltage, temperature, light intensity — by translating continuous analog signals into discrete digital values. The STM32F411RE integrates a 12-bit successive-approximation ADC with up to 16 input channels, providing 4096 quantization levels over a 0–3.3 V input range.

Building on the GPIO driver from Lab 1 and the TIM/PWM driver from Lab 2, this practice integrates a new ADC driver to close a hardware control loop: a potentiometer provides an analog voltage, the ADC digitizes it, and the result is mapped in real time to a PWM duty cycle on TIM1 Channel 1. Rotating the potentiometer continuously varies the brightness of an LED on PA8, demonstrating sensor-actuator feedback without an RTOS or HAL abstraction.

The entire implementation was written in C with CMSIS register definitions, no HAL or CubeMX dependencies, compiled with arm-none-eabi-gcc, and flashed via OpenOCD using the integrated ST-LINK/V2 interface.

II. MATERIALS AND EQUIPMENT

The following hardware and software resources were used:

- NUCLEO-F411RE board (STM32F411RE, Cortex-M4)
- USB-A to Mini-USB cable (power and ST-LINK programming)
- 10 kΩ potentiometer (analog sensor input, wired to PA1 / ADC1_IN1)
- Tektronix oscilloscope (PWM waveform verification on PA8)
- Personal computer, Windows 11
- STM32CubeIDE 2.0.0 (toolchain and OpenOCD only)
- Visual Studio Code with MSYS2/MinGW64 terminal
- arm-none-eabi-gcc cross-compiler
- OpenOCD (flashing via ST-LINK/V2)
- Custom Makefile build system
- CMSIS STM32F411xE device headers (stm32f411xe.h)

III. Process Description

The development followed a bottom-up modular approach. First, the GPIO driver was extended to support analog input mode (GPIO_MODE_ANALOG) for PA1, disabling pull-up/pull-down resistors to avoid interference with ADC conversion. PA8 retained its Alternate Function configuration for TIM1_CH1 from Lab 02.

The ADC driver (adc_driver.h / adc_driver.c) was implemented to configure ADC1 by enabling the APB2 clock via RCC->APB2ENR, setting 12-bit resolution in CR1, selecting single-conversion mode (CR2.CONT = 0), configuring the sampling time in SMPRx, writing the channel sequence in SQR3, setting sequence length to 1 in SQR1.L, and enabling the peripheral via CR2.ADON with a stabilization delay. The adc_start_conversion() function sets CR2.SWSTART, and adc_read_value() polls SR.EOC before reading the 12-bit result from the DR register.

The sensor module wraps the ADC driver with three functions: sensor_init() configures PA1 in analog mode and initializes ADC1 on Channel 1; sensor_startConversion() triggers a single acquisition; and sensor_readValue() returns the raw 12-bit result. The PWM module was reused from Lab 02 without structural changes, with a new pwm_update_duty() helper that updates CCR1 at runtime without stopping the counter. The 12-bit ADC result is mapped to duty cycle using integer arithmetic: duty = raw × 100 / 4095.

Several challenges were encountered during development:

- ADC stabilization timing: Initially, conversions were triggered immediately after CR2.ADON was set, resulting in incorrect readings on the first conversion. The issue was resolved by introducing a software stabilization delay loop after enabling the ADC, as required by the STM32F411RE reference manual.
- Register configuration order: Incorrect ordering of ADC register writes (specifically writing SQR3 before enabling the clock) caused the peripheral to remain in an undefined state. Consulting RM0383 and restructuring the initialization sequence in adc_init() resolved this.
- Analog noise on PA1: At low potentiometer positions, minor voltage fluctuations caused the duty cycle to oscillate slightly around low values. This was mitigated by increasing the ADC sampling time in SMPRx to allow the sample-and-hold capacitor more time to charge.
- Integer truncation in the duty cycle mapping: The expression raw × 100 / 4095 using uint8_t intermediate variables caused overflow. Casting to uint16_t before the multiplication corrected the computation and produced accurate duty cycle percentages.

IV. RESULTS

Both the ADC acquisition and the PWM output functioned correctly on first deployment. Rotating the potentiometer from its minimum to its maximum position produced a continuous and proportional change in LED brightness on PA8, observable by eye. At minimum potentiometer voltage the 12-bit ADC result approached 0, driving the duty cycle to ~0% and extinguishing the LED. At maximum voltage the ADC result reached ~4095, mapping to ~100% duty cycle and producing full LED brightness. The transition across the full sweep was smooth and proportional.

Circuit schematic: The potentiometer was wired with its outer terminals connected to 3.3 V and GND, and its wiper connected to PA1 (ADC1_IN1). The LED on PA8 was driven directly by the TIM1_CH1 PWM output through the onboard current-limiting resistor.

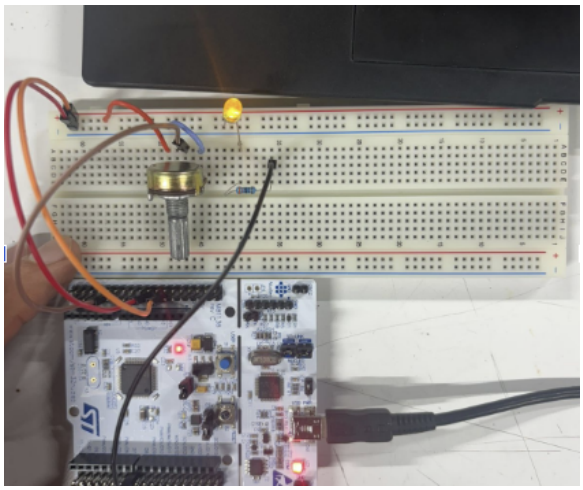

Fig. 1: NUCLEO-F411RE with LED on — potentiometer at maximum position (high duty cycle)

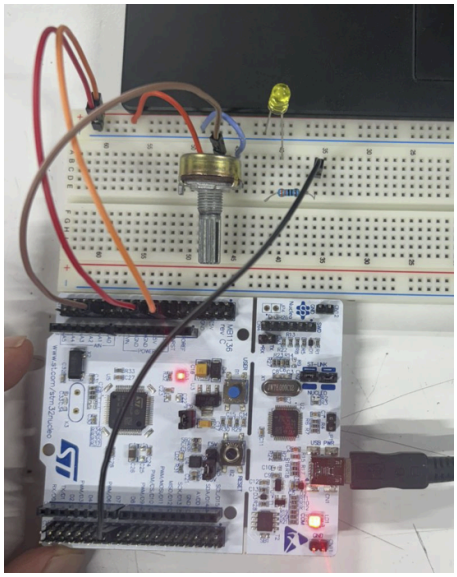

Fig. 2: NUCLEO-F411RE with LED off — potentiometer at minimum position (duty cycle ~0%)

Source code and project files are available in the team's GitHub repository:

1. https://github.com/lopezdaniel-web/MSE-Lab
2. *https://github.com/JPaez725/MSE-Lab*
3. *https://github.com/CrisxCross15/MSE-Lab*
4. *https://github.com/alugobjj/MSE-Lab/tree/main/Lugo*

A video demonstration of the system operating in real time is available at the following link: https://youtube.com/shorts/OlhA2qK2gaI?feature=share

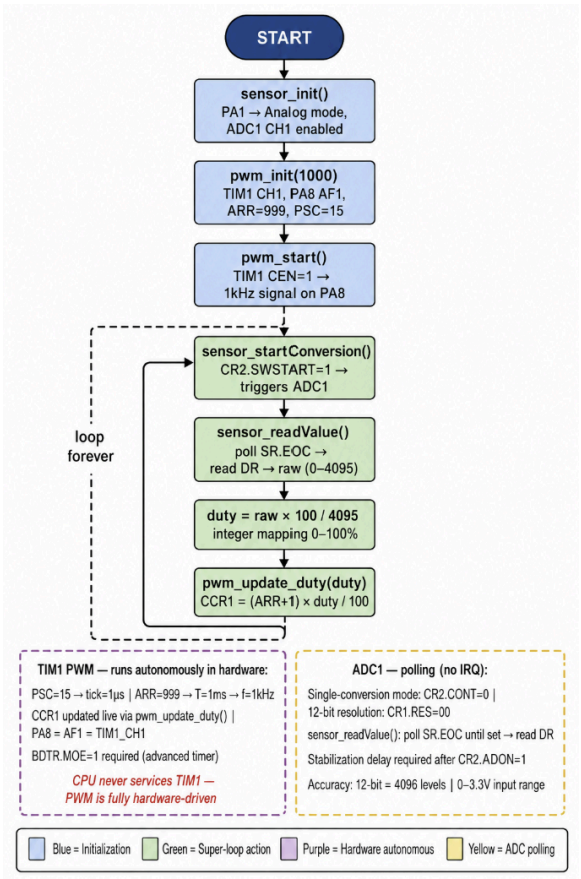

Fig. 3: Diagrama esquemático — potenciómetro de 10 kΩ conectado a PA1 (ADC1_IN1) como entrada analógica y LED onboard LD2 controlado mediante salida PWM en PA8 (TIM1_CH1) sobre la tarjeta NUCLEO-F411RE.

The two hardware states confirmed correct ADC-to-PWM mapping: at minimum potentiometer voltage the 12-bit ADC result approached 0, driving the duty cycle to ~0% and extinguishing the LED. At maximum potentiometer voltage the ADC result reached ~4095, mapping to ~100% duty cycle and driving the LED to full brightness. The transition between both states was smooth and proportional across the full potentiometer sweep.

The Software Design Document (SDD) for the ADC driver is included in the repository under the /SDD directory.

V. CONCLUSION

This laboratory successfully met all stated objectives. An ADC driver was designed and implemented from scratch based on the provided SRS, following functional requirements FR-1 through FR-9 and including error handling for invalid ADC instances and channels. A sensor module was built on top of the ADC driver, providing a hardware-agnostic API for analog readings. The PWM module from Lab 02 was integrated without modification, and the two modules were combined to create a real-time sensor-actuator feedback loop.

The modular architecture (gpio_driver → adc_driver → sensor / tim_driver → pwm → main) proved effective at isolating hardware complexity from application logic. Higher-level modules remained hardware-agnostic and reusable. Hardware validation confirmed correct end-to-end behavior across the full potentiometer sweep.

Through this practice, the team gained hands-on experience with STM32 ADC register-level configuration, the importance of peripheral initialization order and stabilization delays, integer arithmetic precision in embedded mappings, and the value of hardware abstraction through layered driver design. These skills directly reinforce the principles of bare-metal embedded systems development without reliance on HAL or RTOS frameworks.

REFERENCES

[1] STMicroelectronics, "STM32F411xC/E Reference Manual (RM0383)," Rev. 3, 2019.

[2] STMicroelectronics, "NUCLEO-F411RE User Manual (UM1724)," Rev. 14, 2021.

[3] ARM Limited, "Cortex-M4 Devices Generic User Guide (DUI0553)," 2010.

[4] J. Valvano, Embedded Systems: Introduction to ARM Cortex-M Microcontrollers, 5th ed., 2014.